

AUTONOMOUS PARKING PROJECT: INTEGRATION TESTING AND COVERAGE

Dinh Nhat Tien Phan, Crispin Kipruto, Anderson Manuel Briceño Nontol and Romasa Ali.

CONTENT

1. INTRODUCTION	2
1.1. Requirements of Phase 2.....	3
2. FUNCTIONAL REQUIREMENT ANALYSIS.....	3
3. ARCHITECTURAL DESIGN.....	4
4. TEST CASES DESIGN	4
1.2. Test cases design of AutonomousParking methods	4
4.1.1. MoveForward	4
4.1.2. MoveBackward	5
4.1.3. Park.....	6
4.1.4. CheckForMostEfficientFreeSpot	6
1.3. Test cases design of Actuator methods	7
4.1.5. UpOneStep	7
4.1.6. DownOneStep.....	7
1.4. Test cases design of DataSensor methods.....	8
4.1.7. GetDataSensor	8
4.1.8. IsDataInRange	8
4.1.9. FilterNoise.....	9
4.1.10. CalculateData	10
5. TDD OF NEW MODULES	10
1.5. Example 1: TDD for one DataSensor method.....	10
6. INTEGRATING TESTING AND COVERAGE.....	12
1.6. Testing scenario 1:.....	12
1.7. Testing scenario 2:.....	15
1.8. Coverage	17

1. INTRODUCTION

This report presents the implementation of the Autonomous Parking Assistant based on the modules developed during Phase 1 of the project.

The new implementation follows the Test-Driven Development (TDD) paradigm and includes a class diagram illustrating the relationship among the different classes and components involved in the system. Finally, the report presents two complete integration test scenarios designed to evaluate the overall functionality of the application and demonstrate the results of the integration testing process.

1.1. Requirements of Phase 2

- In this phase the module is implemented by making calls to the methods of the class implemented in Phase 1.
- Sensor and Actuator classes are integrated into the class designed in Phase 1.
- Sensors are the ultrasound sensors; in this phase these sensors return random or fixed values according the requirements.
- The controller of the car engine is the Actuator. This will be called in the MoveForward and MoveBackward methods implemented in Phase 1.
- The sensor data should be mocked such that it represents a street with three parking places of mutually different sizes according to the following specifications:
 - One should not be enough for safe parking and the other two enough for parking.
 - One of the sensors should get broken halfway in the middle of the scenario (i.e.k when the car has reached the middle of the street while moving forward) so that it constantly produces recognizable out-of-bound values.

2. FUNCTIONAL REQUIREMENT ANALYSIS

The Table 1 summarizes the functional requirements of this phase of the project.

Req. N°	Name	Type	Function Relevance	Descriptions
R6.5	Move Forward	Functional	MoveForward()	Read a current position before incrementing position
R6.6		Functional	MoveForward()	if sensor data returns valid, increasing one Free Spot Length
R6.7		Functional	MoveForward()	if sensor data returns invalid, reset Free Spot Length
R6.8		Functional	MoveForward()	if sensor data returns invalid Free Spot Length ≥ 5 Current MostEfficientFreeSpot = null Then update MostEfficientFreeSpot
R6.9		Functional	MoveForward()	if sensor data returns invalid Free Spot Length ≥ 5 Current MostEfficientFreeSpot length > Free Spot Length Then update MostEfficientFreeSpot
R6.10		Functional	MoveForward()	if car position = 500 Free Spot Length ≥ 5 Current MostEfficientFreeSpot length > Free Spot Length Then update MostEfficientFreeSpot
R6.11	Parking	Functional	MoveForward()	if car position = 500 Free Spot Length ≥ 5 Current MostEfficientFreeSpot length = 0 Then update MostEfficientFreeSpot
R8.4		Functional	Park()	Keep moving forward until car position = 500
R8.5		Functional	Park()	If none of the MostEfficientFreeSpot is found, return FALSE and update status to UNPARKED
R8.6	Actuators	Functional	Park()	Otherwise keep moving backward until the position of the MostEfficientFreeSpot is found -> update status to PARKED , RESET freeSpotsLength, MostEfficientFreeSpot, and return TRUE
R11.2		Functional	MoveForward()	Increment a car position by 1

R11.3		Functional	MoveBackward()	Decrement a car position by 1
R11.4		Functional	MoveForward(), MoveBackward()	Return the current car position on road

Table 1. Relationships among methods and the requirements they cover.

3. ARCHITECTURAL DESIGN

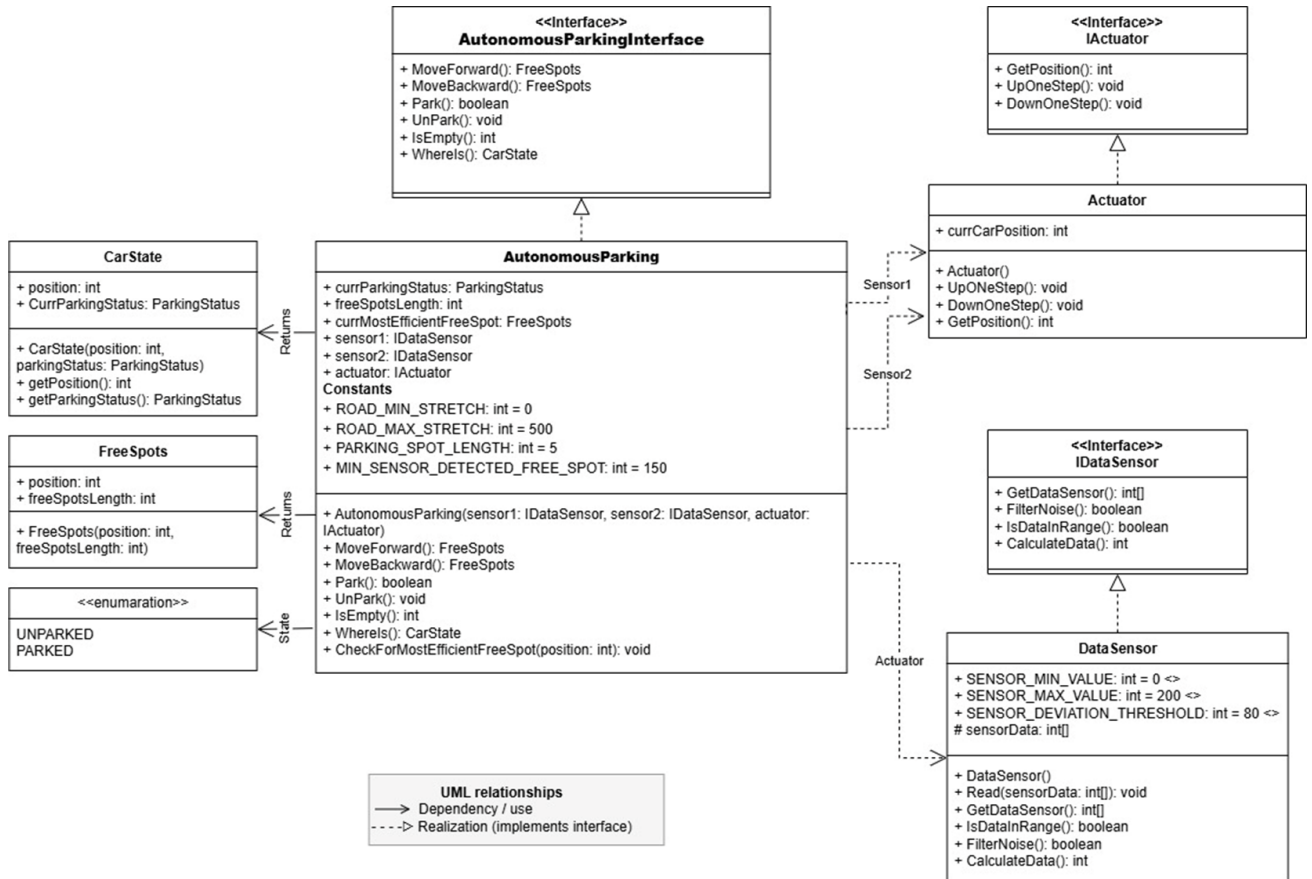


Figure 1. Class Diagram of the Autonomous Parking Assistant module.

4. TEST CASES DESIGN

1.2. Test cases design of AutonomousParking methods

Based on the updated requirements of Phase 2 of the project, some methods have been updated accordingly, these are Park, MoveForward and MoveBackward and a new method has been introduced – CheckForMostEfficientFreeSpot. As a consequence, their specifications and the new test cases are summarized.

4.1.1. MoveForward

4.1.1.1. Requirements

Requirements R6.5, R6.6, R6.7, R6.8, R6.9, R6.10, R6.11, R11.2, R11.4 summarized in Table 1 are covered for this method.

4.1.1.2. Inputs

- Checks if there is free space to the right using “IsEmpty” method.

4.1.1.3. Outputs

- Returns the parkingState (position and freeSpots)

4.1.1.4. Pre-conditions

- carParkingState = UNPARKED

4.1.1.5. Post-conditions

- carParkingState = UNPARKED

4.1.1.6. Test design technique

- Decision Table:

Conditions / Actions	TC-MF-1	TC-MF-2	TC-MF-3	TC-MF-4	TC-MF-5
c1: carParkingState = UNPARKED	True	True	True	True	False
c2: isEmpty() >= 150cm	True	True	False	False	-
c3: position = 499	True	False	True	False	-
a1: wrong input/state	-	-	-	-	X
a2: position += 1, freeSpots = 0	-	-	-	X	-
a3: position += 1, checkMostEfficientSpot(), freeSpots = 0	-	-	X	-	-
a4: position += 1, freeSpots += 1	-	X	-	-	-
a5: position += 1, freeSpots +=1, checkMostEfficientSpot()	X	-	-	-	-

Table 2. Proposed decision table for MoveForward method.

4.1.2. MoveBackward

4.1.2.1. Requirements

Requirements R11.3, R11.4, summarize in Table 1 are covered for this method.

4.1.2.2. Inputs

- Queries for free space to the right using "IsEmpty" method.

4.1.2.3. Outputs

- Returns the parking state of the car (position and freeSpots detected)

4.1.2.4. Pre-conditions

- carParkingState = UNPARKED

4.1.2.5. Post-conditions

- carParkingState = UNPARKED

4.1.2.6. Test design technique

Conditions / Actions	TC-MB-1	TC- MB -2	TC- MB -3
c1: currParkingState = UNPARKED	True	True	False
c2: isEmpty() >= 150 cm	True	False	-
a1: wrong input/state	-	-	X
a2: position -= 1, freeSpots = 0	-	X	-
	X		-

a3: position -= 1, freeSpots += 1		-	
-----------------------------------	--	---	--

Table 3. Decision table for MoveBackward method.

4.1.3. Park

4.1.3.1. Requirements

Requirements R8.4, R8.5, R8.6, summarize in Table 1 are covered for this method.

4.1.3.2. Inputs

- currCarPosition
- currMostEfficientFreeSpot

4.1.3.3. Outputs

- Returns a Boolean value:
 - True: Car is parked successfully, updates parking status to parked.
 - False: Car is not parked successfully, updates parking status to unparked.

4.1.3.4. Pre-conditions

- Car's parking status is currently UNPARKED.
- Car's position is at the start of the street.

4.1.3.5. Post-conditions

- If the most sufficient parking spot of at least 5m is detected:
 - The car moves backward to this spot.
 - The pre-programmed reserve parallel parking maneuver is performed.
 - The car's parking status becomes PARKED.
- If no parking spot is detected at the end of the street:
 - The car's position is 500m.
 - The car's parking status remain UNPARKED.

4.1.3.6. Test design technique

Conditions / Actions	TC-P-1	TC-P-2	TC-P-3
c1: A valid parking stretch found ?	True	False	False
c2: Multiple valid stretches found?	False	True	False
a1: Stay at 500 m	-	-	X
a2: Select valid parking stretch	X	X	-
a3: Move backward to selected spot	X	X	-
a4: final decision PARKED	X	X	-
a5: final decision UNPARKED	-	-	X

Table 4. Decision table for Park method.

4.1.4. CheckForMostEfficientFreeSpot

4.1.4.1. Description

As one of the requirements of the Phase 2 of the project is to park the car in the most space-efficient parking place available, then we introduce this method.

4.1.4.2. Inputs

- Position of the car
- The number of free/empty 1m spots backwards from the provided position.
- Current/Previous most efficient parking space.

4.1.4.3. Outputs

- Updates the current most efficient parking space.

4.1.4.4. Pre-conditions

- Current most efficient parking spot's length is either zero, or ≥ 5 m.

4.1.4.5. Post-conditions

- Updated most efficient parking spot's length is either zero, or ≥ 5 m.

4.1.4.6. Test design technique

Conditions / Actions	TC-CF-1	TC-CF-2	TC-CF-3	TC-CF-4
c1: freeSpotLen > 5m	True	True	True	False
c2: currMostEfficientSpot.length	≥ 5 m	≥ 5 m	0	-
c3: freeSpotsLen < currMostEfficientSpot.freeSpotsLen	True	False	-	-
a1: do nothing	-	X	-	X
a2: currMostEfficientSpot = (position, freeSpotLen)	X	-	X	-

Table 5. Decision table for CheckForMostEfficientFreeSpot method.

1.3. Test cases design of Actuator methods

4.1.5. UpOneStep

4.1.5.1. Inputs

- Checks the current car's position.

4.1.5.2. Outputs

- Updates the car's position.

4.1.5.3. Pre-conditions

- $0 \leq \text{carState.position} \leq 499$

4.1.5.4. Post-conditions

- $1 \leq \text{carState.position} \leq 500$

4.1.5.5. Test design technique

Conditions / Actions	TC-ACU-1	TC-ACU-2
c1: $0 \leq \text{position} \leq 499$	True	False
a1: wrong input/state	-	X
a2: position += 1	X	-

Table 6. Boundary value conditions proposed for UpOneStep method

4.1.6. DownOneStep

4.1.6.1. Inputs

- Checks the current car's position.

4.1.6.2. Outputs

- Updates the car's position.

4.1.6.3. Pre-conditions

- $1 \leq \text{carState.position} \leq 500$

4.1.6.4. Post-conditions

- $0 \leq \text{carState.position} \leq 499$

4.1.6.5. Test design technique

Conditions / Actions	TC-ACD-1	TC-ACD-2
c1: $0 \leq \text{position} \leq 499$	True	False
a1: wrong input/state	-	X
a2: $\text{position} \neq 1$	X	-

Table 7. Boundary value conditions proposed for DownOneStep method

1.4. Test cases design of DataSensor methods

4.1.7. GetDataSensor

4.1.7.1. Description

- Returns the current sensor measurement stored in the sensor.

4.1.7.2. Inputs

- None

4.1.7.3. Outputs

- Returns the current SensorData array.

4.1.7.4. Pre-conditions

- sensorData has been initialized.

4.1.7.5. Post-conditions

- The current sensorData array is returned without modification.

4.1.7.6. Test design technique

Conditions / Actions	TC-SSG-1
c1: sensorData is initialized	True
a1: return sensorData	X

Table 8. One condition to be tested for GetDataSensor method.

4.1.8. IsDataInRange

4.1.8.1. Description

- Checks whether all sensor measurements are within the valid sensor range.
- A sensor value is considered valid when it is between 0 and 200 inclusive.

4.1.8.2. Inputs

- Current sensorData array.

4.1.8.3. Outputs

- Returns true if all sensor measurements are within the valid range.
- Returns false if at least one sensor measurement is outside the valid range.

4.1.8.4. Pre-conditions

- sensorData contains sensor measurement.

4.1.8.5. Post-conditions

- Return's true when every sensor value satisfies:
 - $\text{SENSOR_MIN_VLAUE} \leq \text{sensor value} \leq \text{SENSOR_MAX_VALUE}$
- Returns false when at least one sensor value is below the minimum or above the maximum.

4.1.8.6. Test design technique

Conditions / Actions	TC-SSIDR-1	TC-SSIDR-2	TC-SSIDR-3
c1: sensorData < SENSOR_MIN_VALUE	False	True	False
c2: sensorData > SENSOR_MAX_VALUE	False	False	True
a1: return false	-	X	X
a2: return true	X	-	-

Table 9. Decision table proposed for IsDataInRange method.

4.1.9. FilterNoise

4.1.9.1. Description

- Determines whether the sensor data is noise or not.
- The difference between the maximum and minimum sensor values is used to detect abnormal noise.

4.1.9.2. Inputs

- Current sensorData array.

4.1.9.3. Outputs

- Returns true if the deviation between the maximum and minimum values Is less than or equal to SENSOR_DEVIATION_THRESHOLD.
- Returns false if the deviation is greater than SENSOR_DEVIATION_THRESHOLD.

4.1.9.4. Pre-conditions

- sensorData contains sensor measurements.

4.1.9.5. Post-conditions

- maxValData contains the maximum value in sensorData.
- minValData cotains the minimum value in sensorData.

- Returns true when:
 $\text{maxValData} - \text{minValData} \leq \text{SENSOR_DEVIATION_THRESHOLD}$
- Returns false when:
 $\text{maxValData} - \text{minValData} > \text{SENSOR_DEVIATION_THRESHOLD}$

4.1.9.6. Test design technique

Conditions / Actions	TC-SSFN-1	TC-SSFN-2	TC-SSFN-3
c1: $\text{maxValData} - \text{minValData} > 80$	False	True	False
c2: $\text{maxValData} - \text{minValData} = 80$	True	-	False
a1: return false	-	X	-
a2: return true	X	-	X

Table 10. Decision table proposed for FilterNoise method.

4.1.10. CalculateData

4.1.10.1. Description

- Calculates the average value of all sensor measurements.

4.1.10.2. Inputs

- Current sensorData array.

4.1.10.3. Outputs

- Returns the arithmetic mean of all sensor measurements.

4.1.10.4. Pre-conditions

- sensorData contains at least one sensor measurement.

4.1.10.5. Post-conditions

- The returned value is the integer average of all values in sensorData.
- The original sensorData array is not modified.

4.1.10.6. Test design technique

Conditions / Actions	TC-SSCD-1
c1: sensorData contains valid values	True
a1: return integer average	X

Table 11. One condition to be tested for CalculateData method.

5. TDD OF NEW MODULES

For the sake of simplicity, just two examples of the use of Test-Driven Development paradigm are shown in this chapter. One that belongs to the Actuator class and other for the DataSensor class.

1.5. Example 1: TDD for one DataSensor method

- Figure 2 shows the tags used to relate each test case to the Test Design Technique.
- Figure 3 exhibits the first implementation of the CalculateData method. It contains an error.

- Figure 4 shows the result of the executed test. It failed.
- Figure 5 highlights the found error and its correction.
- Figure 6 the test was run again and this time it passed.

Testing tag



```
@Test // TC_SSCD_1
void TestCalculateData() {
    int[] sensorData1 = {160, 100, 102, 104, 150};
    int[] sensorData2 = {122, 98, 102, 104, 150};

    IDataSensor sensor1 = new DataSensor();
    IDataSensor sensor2 = new DataSensor();

    sensor1.Read(sensorData1);
    sensor2.Read(sensorData2);

    assertEquals( expected: 123, sensor1.CalculateData());
    assertEquals( expected: 115, sensor2.CalculateData());
}
```

Figure 2. Tag used to relate the test case with the Test Design Technique selected.

```
public int CalculateData() {

    int sum = 0;

    for (int i = 0; i < this.sensorData.length; i++) {
        sum += (this.sensorData[i]);
    }

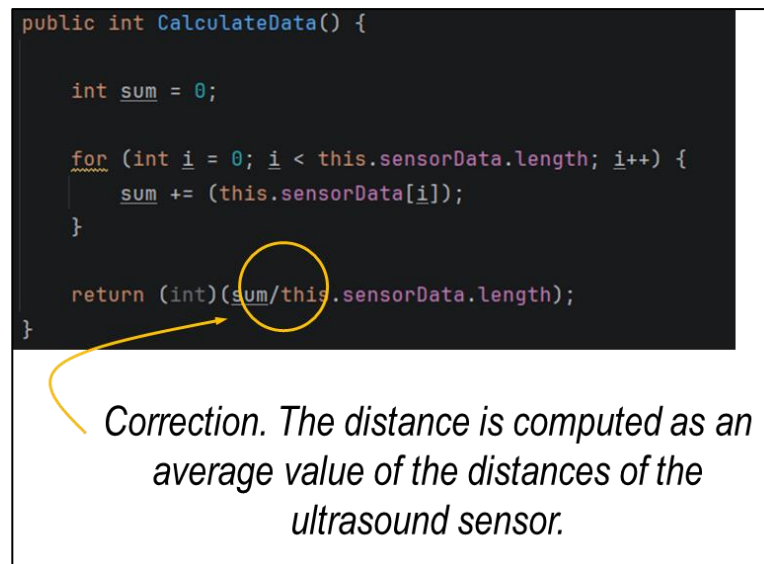
    return (int)(sum*this.sensorData.length);
}
```

Figure 3. First implementation of CalculateData method.



```
TestDataSensor 1 sec 327 ms
TestCalculateData() 41 ms
"C:\Program Files\Java\jdk-21.0.12.1\bin\java.exe" ...
org.opentest4j.AssertionFailedError:
Expected :123
Actual   :3080
```

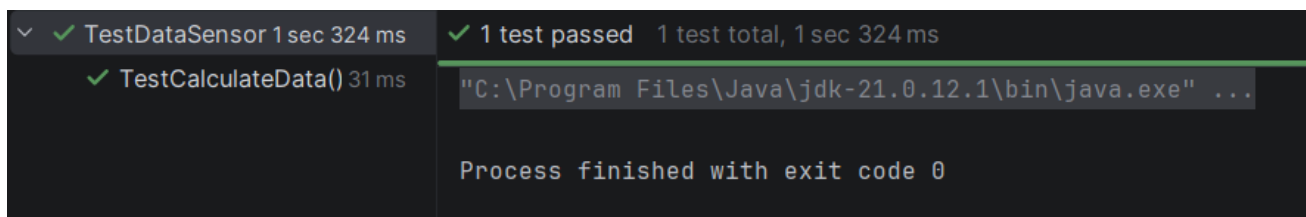
Figure 4. The first test run results in an error.



```
public int CalculateData() {
    int sum = 0;
    for (int i = 0; i < this.sensorData.length; i++) {
        sum += (this.sensorData[i]);
    }
    return (int)(sum/this.sensorData.length);
}
```

Correction. The distance is computed as an average value of the distances of the ultrasound sensor.

Figure 5. Parsing the code, we realize that there was an error in the implementation.



```
TestDataSensor 1 sec 324 ms
TestCalculateData() 31 ms
"C:\Program Files\Java\jdk-21.0.12.1\bin\java.exe" ...
Process finished with exit code 0
```

Figure 6. Test after the performed correction.

6. INTEGRATING TESTING AND COVERAGE

1.6. Testing scenario 1:

As stated in the project description for phase 2.

- Start at the beginning of the street
- Move along the street and scan the available parking places.
- Move backwards until the most efficient parking place (the smallest available parking where it can still park safely),
- Park the car.
- Unpark the car and drive to the end of the street.

The implemented test code for this scenario is shown in Figure 7. The next lines describe the details of the implementation.

- A) We start by defining the simulated sensors measurements associated with the different conditions: FREE, BLOCKED and OUT-OF-RANGE.
- B) We define three spots where the sensors will take measurements corresponding to FREE space. These spots are better represented in Figure 8.
- C) We initialize the variable randomSensorId, which randomly selects the sensor that gets broken.
- D) The variable randomPosition is initialized with a random position where the sensor gets broken.
- E) This block of code initializes the measurements for sensor 1 based on the previously defined information. An identical block of code is implemented for sensor 2, but it is not shown in the figure.
- F) The test code is implemented.
 - f1) An instance of the AutonomousParking class named **car** is initialized.
 - f2) the **Park** method is executed on the **car** instance. ***As required, the car must move forward to the end of the street while scanning the available free parking spots. It must then move forward to the most space-efficient parking spot identified and park there.***

Based on the requirements, the car must park between 254m and 258m, as this is the most space-efficient parking spot with a length of 5 meters.

- f3) This line asserts that the car is parked.
- f4) This line asserts that the car is parked between the spaces 254m and 258m.

As required, after the car is parked, it must unpark and move to the end of the street.

- f5) After the car is parked, the **UnPark** method is executed.
- f6) This line asserts that the car is unparked.
- f7) Moves the car to the end of the street.
- f8) This line asserts that the car is at the end of the street.

```

public class TestScenario_1 {
    private static Actuator actuator = new Actuator();

    private static DataSensor sensor1 = spy(new DataSensor());
    private static DataSensor sensor2 = spy(new DataSensor());

    a private static final int[] FREE = {180, 180, 180, 180, 180};
    private static final int[] BLOCKED = {100, 100, 100, 100, 100};
    private static final int[] OUT_OF_RANGE = {-1, 220, 10, 500, 20};

    @BeforeAll
    static void setupParkingStreet() {
        /**
         * Let's set up our free parking stretches as below:
         * - 11 to 13: 3 free spots (not enough)
         * - 101 to 108: 8 spots free (enough)
         * - 254 to 258: 5 spots free (enough)
         */
        b ValueRange spot1 = ValueRange.of(11, 13);
        ValueRange spot2 = ValueRange.of(101, 108);
        ValueRange spot3 = ValueRange.of(254, 258);

        /* Pick one sensor (randomly) to return out-of-range-values (from a random
        range measurements
        int randomSensorId = (int) (Math.random() * 2);
        int randomPosition = (int) ((Math.random() * 500) + 1);
        c d Variable to randomly select the sensor with out-of-
        range measurements
        Variable used to randomly locate the point where one of
        the sensors gets broken.

        /* Set up mock values for sensor 1 */
        e when(sensor1.getDataSensor()).thenAnswer(InvocationOnMock invocation -> {
            int position = actuator.getPosition();
            if (randomSensorId == 0 && position >= randomPosition) {
                /* Use out of range values */
                sensor1.read(OUT_OF_RANGE);
            }
            else if (spot1.isValidValue(position) || spot2.isValidValue(position) || spot3.isValidValue(position)) {
                sensor1.read(FREE);
            } else {
                sensor1.read(BLOCKED);
            }
            return invocation.callRealMethod();
        });

        f @Test
        void testCompleteParkingAndUnparking() {
            f1 AutonomousParking car = new AutonomousParking(sensor1, sensor2, actuator);

            /* Call the parking method */
            f2 assertDoesNotThrow(() -> car.park());
            f3 assertEquals(car.whereIs().getParkingStatus(), ParkingStatus.PARKED);
            f4 assertEquals(car.whereIs().getPosition(), actual: 258);

            /* Test unparking and move to end of street */
            f5 assertDoesNotThrow(() -> car.unPark());
            f6 assertEquals(car.whereIs().getParkingStatus(), ParkingStatus.UNPARKED);
            f7 while(car.whereIs().getPosition() < AutonomousParking.ROAD_MAX_STRETCH) {
                assertDoesNotThrow(() -> car.moveForward());
            }
            f8 assertEquals(car.whereIs().getPosition(), AutonomousParking.ROAD_MAX_STRETCH);
        }
    }
}

```

Figure 7. Implemented code to test scenario 1.

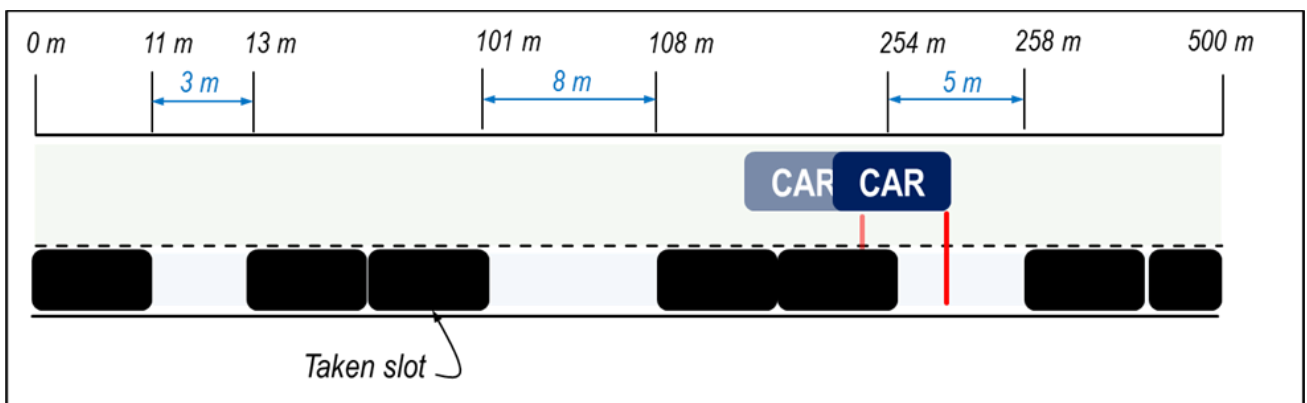


Figure 8. Graphic representation of the predefined spots with free space for parking.

1.7. Testing scenario 2:

- Start at the beginning of the street.
- Move along the street and scans available parking places.
- There are three parking places that are insufficient for parking. (less than 5 meters)
- Reach the end of the street without parking.
- The car remains unparked.

The implemented test code for this scenario is shown in Figure 9 and Figure 10. The next lines describe the details of the implementation.

- A) We start by defining the simulated sensors measurements associated with conditions such as “free to park” (FREE), “obstacle presence” (BLOCKED), noisy data (NOISE) and out-of-range data (OUTOFRANGE). These four arrays could be considered the building blocks used to generate the 500 measurements of each sensor.
- B) This block of code will be used later in the implementation to retrieve data from an “array of measurements”. Each measurement consists of five independent measurements and is associated with a particular position.
- C) This block is used to generate 500 measurements by combining the four different basic sensor measurements defined in a).
- D) One actuator instance is initialized.
- E) These two blocks generate the 500 simulated measurements for each ultrasound sensor. They use the arrays defined in a) and the function defined in c).
- F) The test code is implemented:
 - F1) An instance of **IDataSensor** named **sensor1** is initialized using the block of code defined in c)
 - F2) An instance of **IDataSensor** named **sensor2** is initialized using the block of code defined in c).
 - F3) An instance of **AutonomousParking** class named **car** is initialized.
 - F4) The **Park** method is executed on the **car** instance.

The requirements state that the car must move forward to the end of the street while scanning the available parking places. However, this test scenario does not include any parking spot that meets the minimum required length of meters. Therefore, once the car has reached the end, it remains there and its parking status must be UNPARKED.

F5) On this line the position is retrieved by the **WhereIs** method.

F6) This line asserts that the car’s position is 500.

F7) this line asserts that the car’s parking status is not PARKED.

F8) This line asserts that the car’s parking status is UNPARKED.

```

class TestScenario_2 {
    a
    private static final int[] FREE = {180, 180, 180, 180, 180};

    private static final int[] BLOCKED = {100, 100, 100, 100, 100};

    private static final int[] NOISE = {150, 100, 100, 100, 10};

    private static final int[] OUTFRANGE = {-1, 220, 10, 500, 20};

    b
    private IDataSensor createMockSensor(int[]... sequence)
    {
        IDataSensor sensor = spy(new DataSensor());

        AtomicInteger index = new AtomicInteger( initialValue: 0);

        int[][] currentData = {sequence[0]};

        when(sensor.GetDataSensor())
        .thenAnswer( InvocationOnMock invocation -> {
            int i = index.getAndIncrement();

            if (i < sequence.length) {
                return currentData[0] = sequence[i];
            }

            return invocation.callRealMethod();
        });

        return sensor;
    }

    c
    private int [][] createRoadConditions(
        int [][] free,
        int [][] noise,
        int [][] ofoutrange
    )
    {
        int roadLength = 500;
        int[][] roadConditions = new int[roadLength][5];

        // Default everything to BLOCKED
        for (int i = 0; i < roadLength; i++) {
            roadConditions[i] = BLOCKED.clone();
        }

        // FREE ranges
        for (int[] range : free) {
            for (int pos = range[0]; pos <= range[1]; pos++) {
                roadConditions[pos - 1] = FREE.clone();
            }
        }

        // NOISE ranges
        for (int[] range : noise) {
            for (int pos = range[0]; pos <= range[1]; pos++) {
                roadConditions[pos - 1] = NOISE.clone();
            }
        }

        // of out range ranges
        for (int[] range : ofoutrange) {
            for (int pos = range[0]; pos <= range[1]; pos++) {
                roadConditions[pos - 1] = OUTFRANGE.clone();
            }
        }

        return roadConditions;
    }
}

```

Predefined simulated sensor measurement sets

This block will retrieve simulated sensor measurements from the given sequence array

This block generates 500 arrays of measurements by combining the predefined FREE, BLOCKED, NOISE, and OUTFRANGE data.

Figure 9. First part - Implemented code for test scenario 2.

```

@fetest
void testCouldNotFindAnySpots() {

    /* Starting at the beginning of the street */
    IActuator actuator = new Actuator();

    /* Encounters 3 parking spaces that are insufficient safe parking */
    int[][] road_1 = createRoadConditions(
        new int[][] { // Free blocks
            {120, 123}, // 4
            {201, 203}, // 4
            {427, 429}, // 3
        },
        new int[][] { // Noise blocks
            {1, 2}, // 2
            {20, 23}, // 4
            {99, 110}, // 12
        },
        new int[][] { // Out of range blocks
            {20, 25}, // 6
        }
    );

    int[][] road_2 = createRoadConditions(
        new int[][] { // Free blocks
            {120, 123}, // 4
            {201, 203}, // 4
            {427, 429}, // 3
        },
        new int[][] { // Noise blocks
            {1, 2}, // 2
            {20, 23}, // 4
            {99, 110}, // 12
        },
        new int[][] { // Out of range blocks
            {20, 25}, // 6
        }
    );

    IDataSensor sensor1 = createMockSensor(road_1);
    IDataSensor sensor2 = createMockSensor(road_2);
    AutonomousParkingInterface car = new AutonomousParking(sensor1, sensor2, actuator);

    /* Does not find any parking space with at least 5 meters of continuous free space */
    boolean parked = car.Park();

    CarState carState = car.WhereIs();

    /* Reaches the end of the street without parking */
    assertEquals("expected: 500, carState.getPosition()");

    /* Remains unparked */
    assertEquals("expected: false, parked");
    assertEquals(ParkingStatus.UNPARKED, carState.getParkingStatus());
}

```

The instance *actuator* is initialized.

This blocks generates the simulated measurement data for each sensor.

Sensor and AutonomousParking instances initialization and assertion tests

Figure 10. Second Part – Implemented code for test scenario 2.

1.8. Coverage

▼  AutonomousParking		100.0 %	303	0	303
 AutonomousParking(IDataSensor,		100.0 %	25	0	25
▲ CheckForMostEfficientFreeSpot(in		100.0 %	24	0	24
● IsEmpty()		100.0 %	74	0	74
● MoveBackward()		100.0 %	40	0	40
● MoveForward()		100.0 %	56	0	56
● Park()		100.0 %	43	0	43
● UnPark()		100.0 %	20	0	20
● WhereIs()		100.0 %	21	0	21

Figure 2. Coverage report for AutonomousParking class.